

Contiguous Memory, clone vs view, Worked Out by Hand

Who shares storage, who copies, and when a copy is forced

A complete numerical walk-through of the one question every tensor bug eventually reduces to: *do these two tensors share the same bytes?* We define C-contiguity precisely, test it on a $2 \times 3 \times 4$ block and its transpose, watch a `view` silently mutate its parent, watch `clone` refuse to, see what `detach` cuts and what it keeps, and force a real copy with `contiguous()`. Nothing is hand-waved — every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It takes the same tiny $2 \times 3 \times 4$ tensor filled with $0, 1, \dots, 23$ used in the Tensors topic, plus a handful of length-3 and length-6 vectors, and tracks the *storage pointer* through `view`, `reshape`, `clone`, `detach` and `contiguous()`. For each it answers four yes/no questions: does it share storage, does it copy, does it change strides, does it break the autograd graph.

How to read this. Each operation gets its own worked step with before/after numbers on the smallest tensor that proves the point, and a coloured box isolating the trap or the right-tool decision. Read this directly after the Tensors and Reshape/View/Permute topics; it formalises the contiguity test those topics introduced.

Concepts covered, in order: the exact C-contiguity test $s_a = \prod_{b>a} d_b$ · verifying it TRUE for $(2, 3, 4)$ and FALSE for its transpose · views and slices share storage (mutation leaks) · `clone` as an independent copy · `detach` sharing storage but cutting the graph · `contiguous()` repacking bytes into storage order · memory accounting $\text{numel} \times \text{itemsize}$ · a full comparison table · why some kernels demand contiguous input.

1 The setup: one storage, many recipes

A PyTorch tensor is two things glued together: a flat run of numbers in memory (the *storage*), and a recipe (shape, stride, offset) for reading that run as an n -dimensional box. Two tensors *share storage* when they point at the same flat run — then writing through one is visible through the other. The entire `clone-vs-view` question is: which operations hand you a new recipe over the *same* storage, and which allocate a *fresh* storage.

We reuse the running $2 \times 3 \times 4$ tensor, fixed to `arange(24)` so each value *is* its own flat offset:

$$T[0] = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{bmatrix}, \quad T[1] = \begin{bmatrix} 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 \\ 20 & 21 & 22 & 23 \end{bmatrix}.$$

Quantity	Symbol	Value here
Shape (size per axis)	(d_0, d_1, d_2)	$(2, 3, 4)$
Number of elements	<code>numel</code>	$2 \cdot 3 \cdot 4 = 24$
Strides (elements)	(s_0, s_1, s_2)	$(12, 4, 1)$
Element type	<code>dtype</code>	<code>float32</code> (4 bytes each)
Memory order	—	row-major (C-contiguous)

Intuition

Think of storage as a single rented warehouse aisle of numbered slots. `view`, `reshape`, slicing, `transpose` and `detach` all hand you a new *floor-plan* over the *same* aisle — cheap, instant, and dangerous, because a write through one floor-plan changes the slots everyone else reads. `clone` rents a brand-new aisle and copies the numbers across — expensive, but now your writes are yours alone. `contiguous()` is the special case of renting a new aisle only when the old floor-plan walks the slots out of order.

2 Step 1 — The exact contiguity test

A tensor is **C-contiguous** when walking it in logical order (last axis fastest) visits storage slots $0, 1, 2, \dots$ with no gaps and no jumps. That happens precisely when every stride equals the product of the sizes of the axes after it:

$$\text{C-contiguous} \iff s_a = \prod_{b>a} d_b \quad \text{for every axis } a$$

This is a per-axis check; the tensor is contiguous only if *all* axes pass.

Test on T , shape $(2, 3, 4)$, strides $(12, 4, 1)$.

$$\begin{aligned} a = 2: \quad s_2 = 1 &\stackrel{?}{=} \prod_{b>2} d_b = 1 && \checkmark \\ a = 1: \quad s_1 = 4 &\stackrel{?}{=} d_2 = 4 && \checkmark \\ a = 0: \quad s_0 = 12 &\stackrel{?}{=} d_1 d_2 = 3 \cdot 4 = 12 && \checkmark \end{aligned}$$

All three axes pass, so `T.is_contiguous()` is **True**.

Test on the transpose (swap axes 1 and 2): shape becomes (2, 4, 3) and the strides swap with the axes to (12, 1, 4) — the bytes do not move. Now run the same test against the *new* shape:

$$\begin{aligned} a = 2 : \quad s_2 = 4 &\stackrel{?}{=} 1 && \times \\ a = 1 : \quad s_1 = 1 &\stackrel{?}{=} d_2 = 3 && \times \\ a = 0 : \quad s_0 = 12 &\stackrel{?}{=} d_1 d_2 = 4 \cdot 3 = 12 && \checkmark \end{aligned}$$

Two axes fail, so `is_contiguous()` is **False**. One failed axis is enough; here axis 2 has stride 4 but a packed last axis would need stride 1.

Mini-example — this operation in isolation

The course’s flat example: `x = arange(12).reshape(3,4)` has strides (4, 1) — contiguous, since $s_1 = 1$ and $s_0 = d_1 = 4$. Its transpose `x.T` has shape (4, 3) and strides (1, 4). Test: $s_1 = 4 \neq 1$, fail $\Rightarrow$ non-contiguous. Transpose never copies; it reverses the stride pair and lets the contiguity test fail.

3 Step 2 — A view shares storage: mutation leaks

`view` and slicing return a new recipe over the *same* storage. The proof is that the storage pointer (`data_ptr`) is identical, and a write through the view appears in the parent.

View. Let $x = [1, 2, 3, 4, 5, 6]$ and $v = x.view(2, 3)$:

$$v = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad x.data_ptr() = v.data_ptr() \Rightarrow \text{shared.}$$

Element $v[0, 1]$ sits at flat offset $0 \cdot 3 + 1 \cdot 1 = 1$ — the same slot as $x[1]$. Writing $v[0, 1] = 99$ therefore writes slot 1 of the shared storage:

$$v = \begin{bmatrix} 1 & \mathbf{99} & 3 \\ 4 & 5 & 6 \end{bmatrix} \implies x = [1, \mathbf{99}, 3, 4, 5, 6].$$

The parent x changed, even though we never named it on the left-hand side.

Slice. The same holds for slices, which only shift the storage offset. Let $y = [10, 20, 30, 40]$ and $s = y[1:3] = [20, 30]$ with `storage_offset = 1`. Writing $s[0] = -7$ hits slot 1 of y :

$$y = [10, \mathbf{-7}, 30, 40].$$

Watch out

This is the single most common silent data-corruption bug. You slice a row out of a batch “to look at it,” normalise it in place with `row -= row.mean()`, and you have just edited the batch your model will train on — with no error, no warning. If you need a row you can safely scribble on, take `row = batch[i].clone()`. A view is a window, not a photocopy.

4 Step 3 — `clone`: an independent copy

`clone` allocates fresh storage and copies the numbers across, so the pointers differ and writes do not leak. With $a = [1, 2, 3]$ and $b = a.clone()$:

$$a.data_ptr() \neq b.data_ptr() \Rightarrow \text{separate storage.}$$

Writing $b[0] = 99$ touches only b 's aisle:

$$b = [99, 2, 3], \quad a = [1, 2, 3] \text{ (unchanged).}$$

This is the behaviour the course summarises as “`x.clone()` → copies memory: Yes.” The cost is real: a clone of T allocates a second $24 \times 4 = 96$ -byte storage, so the two tensors together hold $2 \times 96 = 192$ bytes.

When this is the right choice

Reach for `clone` when you need a value you will mutate in place but must not let that mutation flow back to the source — editing one row of a batch, building an augmented copy, caching a snapshot of weights before an optimiser step. If you also want the copy excluded from autograd (logging, metrics, saving to disk), chain `x.clone().detach()`; the clone makes it independent in memory, the detach makes it independent of the graph.

5 Step 4 — `detach`: shares storage, cuts the graph

`detach` is the subtle one: it returns a tensor over the *same* storage but *outside* the autograd graph. So it is the opposite trade-off from `clone`. With $w = [1, 2, 3]$, `requires_grad=True`, and $d = w.detach()$:

$$w.data_ptr() = d.data_ptr() \Rightarrow \text{shared storage}, \quad w.requires_grad = \text{True}, \quad d.requires_grad = \text{False}.$$

Because the storage is shared, writing through d *still* mutates w 's data — and because d is off the graph, doing so can corrupt the values autograd relies on. By contrast $c = w.clone()$ keeps the graph (`c.requires_grad` is True and c carries a `grad_fn`) but gives fresh storage.

	<code>w.clone()</code>	<code>w.detach()</code>
New storage (copies bytes)?	Yes	No (shared)
On the autograd graph?	Yes (<code>grad_fn</code> set)	No (<code>requires_grad=False</code>)
Writes leak to w 's data?	No	Yes

Watch out

`detach` does *not* make a tensor safe to edit — it only hides it from the gradient. The bytes are still w 's bytes. For a snapshot you can both edit *and* keep off the graph, you need *both*: `w.detach().clone()`. Using `detach` alone and then writing in place is a classic way to poison a training run with no error message.

6 Step 5 — `contiguous()`: repack into storage order

`contiguous()` returns a tensor with the *same logical values* but laid out in packed C-order. If the tensor is already contiguous it is a no-op (returns the same storage); if not, it allocates fresh storage and copies the numbers in walk-order.

Take the transpose from Step 1: shape (2, 4, 3), strides (12, 1, 4), non-contiguous. Walking it in logical order reads the original flat line out of order. Reading the first 12 logical elements of the transpose pulls slots in the pattern 0, 4, 8, 1, 5, 9, ..., which `contiguous()` writes down densely into a brand-new storage:

new packed storage (first 12) = [0, 4, 8, 1, 5, 9, 2, 6, 10, 3, 7, 11].

The logical values are identical to the transpose (`torch.equal` is `True`), but the strides are now the packed ones for shape (2, 4, 3):

strides (12, 1, 4) $\xrightarrow{\text{.contiguous()}}$ (12, 3, 1), $s_1 = d_2 = 3$, $s_0 = d_1 d_2 = 4 \cdot 3 = 12$.

Now the contiguity test passes on every axis, and `is_contiguous()` is `True`.

Mini-example — this operation in isolation

Why call it at all? `view` refuses to operate on non-contiguous storage, because a zero-copy reshape would have to span “two contiguous subspaces.” On the transpose `Tt.view(2,12)` raises `RuntimeError: view size is not compatible with input tensor's size and stride`. The safe pattern is `Tt.contiguous().view(2,12)` — or just `Tt.reshape(2,12)`, which silently calls `contiguous()` for you and returns fresh storage with strides (12, 1).

7 Step 6 — The comparison table

Four yes/no questions decide everything. The answers depend on *both* the operation and whether its input is contiguous. Numbers below are measured: on the contiguous base T , both `reshape(6,4)` and `view(6,4)` keep the storage pointer and produce strides (4, 1); on the non-contiguous transpose, `reshape(2,12)` is forced to copy (new pointer, strides (12, 1)) and `view` raises.

Operation	Shares storage?	Copies?	Changes strides?	Breaks graph?
<code>y = x</code>	Yes	No	No	No
<code>x.view(...)</code>	Yes	No	Yes (new recipe)	No
<code>x.reshape(...)</code> (contig.)	Yes	No	Yes	No
<code>x.reshape(...)</code> (non-contig.)	No	Yes	Yes	No
<code>x[1:3]</code> (slice)	Yes	No	offset/stride	No
<code>x.transpose(...)</code>	Yes	No	Yes (swapped)	No
<code>x.contiguous()</code> (contig.)	Yes	No	No	No
<code>x.contiguous()</code> (non-contig.)	No	Yes	Yes (repacked)	No
<code>x.detach()</code>	Yes	No	No	Yes
<code>x.clone()</code>	No	Yes	No	No
<code>x.clone().detach()</code>	No	Yes	No	Yes

Intuition

Read the table as two independent switches. The *storage* switch (share vs copy) is about memory safety: only `clone` and a *forced contiguous/reshape* flip it to “copy.” The *graph* switch (keep vs break) is about gradients: only `detach` flips it. `clone().detach()` throws both switches; a bare `view` throws neither. Every row is just some combination of these two orthogonal switches.

8 Step 7 — Memory accounting

The cost of “copy” is exact. A tensor’s storage occupies

$$\text{bytes} = \text{numel} \times \text{itemsize}$$

For T : `numel` = 24, `float32` so `itemsize` = 4, giving $24 \times 4 = 96$ bytes. A `view`, `detach`, `transpose` or `slice` adds *zero* new element storage — they reuse the same 96 bytes. A `clone`, or a `contiguous()` on a non-contiguous tensor, allocates a second 96-byte storage, so the live total becomes $2 \times 96 = 192$ bytes.

Result of ...	New element bytes	Live total (with T)	Why
<code>T.view(6,4)</code>	0	96	shares T ’s storage
<code>T.transpose(1,2)</code>	0	96	swaps strides only
<code>T.detach()</code>	0	96	shares storage, off graph
<code>T.clone()</code>	96	192	fresh storage, copied
<code>Tt.contiguous()</code>	96	192	repacks into new storage

Watch out

Some kernels and library calls demand contiguous input and will either error or silently call `contiguous()` for you, doubling memory for that tensor at exactly the wrong moment (a large activation, mid-training). If a model mysteriously needs more memory after you add a `permute`, it is usually a downstream op forcing a repack. Profile, then either reorder so the `permute` is the last thing before a copy you wanted anyway, or accept the explicit `contiguous()` and budget for it.

9 Equation cheat-sheet

Contiguity test	C-contiguous $\iff s_a = \prod_{b>a} d_b$ for all a
Packed strides for shape d	$s_{n-1} = 1, \quad s_a = d_{a+1} s_{a+1}$
Index $\rightarrow$ offset	$\text{offset} = \text{storage_offset} + \sum_a s_a \text{idx}_a$
Storage footprint	$\text{numel} \times \text{itemsize}$
Shares storage?	<code>a.data_ptr() == b.data_ptr()</code>
<code>view/slice/transpose/detach</code>	share storage (no element copy)
<code>clone</code>	fresh storage (copy); keeps graph
<code>detach</code>	same storage; cuts graph
<code>contiguous()</code>	no-op if contiguous, else copy to packed order
<code>reshape</code>	<code>view</code> if contiguous, else <code>contiguous().view</code> (copy)

Appendix A Reference implementation

Every number above is reproducible from the following program (PyTorch 2.x; NumPy gives identical layout). The `data_ptr()` comparisons are the ground truth for “shares storage?”.

```
import torch
from math import prod

# ---- Step 1: the exact contiguity test -----
T = torch.arange(24, dtype=torch.float32).reshape(2, 3, 4)
def c_strides(shape):
    n = len(shape)
    return tuple(prod(shape[b] for b in range(a+1, n)) for a in range(n))

print(T.stride(), T.is_contiguous())          # (12, 4, 1) True
print(c_strides((2,3,4)))                     # (12, 4, 1) -> all axes match

Tt = T.transpose(1, 2)                        # shape (2,4,3)
print(Tt.stride(), Tt.is_contiguous())         # (12, 1, 4) False
print(c_strides((2,4,3)))                     # (12, 3, 1) -> axes 1,2 fail

# ---- Step 2: a view (and a slice) shares storage -----
x = torch.tensor([1., 2., 3., 4., 5., 6.])
v = x.view(2, 3)
print(x.data_ptr() == v.data_ptr())           # True (shared)
v[0, 1] = 99.
print(x)                                       # tensor([ 1., 99.,  3.,  4.,  5.,  6.])

y = torch.tensor([10., 20., 30., 40.])
s = y[1:3]
print(s.storage_offset())                     # 1
s[0] = -7.
print(y)                                       # tensor([ 10., -7., 30., 40.])

# ---- Step 3: clone is an independent copy -----
a = torch.tensor([1., 2., 3.])
b = a.clone()
print(a.data_ptr() == b.data_ptr())           # False (separate)
b[0] = 99.
print(a)                                       # tensor([1., 2., 3.]) (unchanged)

# ---- Step 4: detach shares storage but cuts the graph -----
w = torch.tensor([1., 2., 3.], requires_grad=True)
d = w.detach()
print(w.data_ptr() == d.data_ptr())           # True (shared)
print(w.requires_grad, d.requires_grad)       # True False
c = w.clone()
print(c.requires_grad, c.grad_fn is not None) # True True

# ---- Step 5: contiguous repacks into storage order -----
Tc = Tt.contiguous()
print(Tc.stride(), Tc.is_contiguous())         # (12, 3, 1) True
print(torch.equal(Tc, Tt))                     # True (same values)
print(Tc.flatten()[:12].tolist())
# [0., 4., 8., 1., 5., 9., 2., 6., 10., 3., 7., 11.]
try:
    Tt.view(2, 12)
except RuntimeError as e:
```

```
    print("view failed:", e)                # not compatible with stride
print(Tt.reshape(2, 12).stride())          # (12, 1)  (fresh storage)

# ---- Step 7: memory accounting -----
print(T.numel(), T.element_size(),
      T.numel() * T.element_size())        # 24 4 96
print(2 * T.numel() * T.element_size())   # 192  (T + one clone)
```

— end of document —